Date __/__/__
Intimate

— SEGMENT TREE —

3	1	2	7	5	6	3

index : (1,4) → sum →
index : (2,5) → sum →
index : (0,5) → sum →

$O(Query * N)$

$Q = 10^4$, large $n \rightarrow 10^6$

$O(10^4 * 10^6)$

Segment Tree is an efficient data structure that allows -

⇒ Efficient Querying of intervals / range.
⇒ Efficient updating of intervals / range.

range queries to find

↳ Sum

↳ Minimum

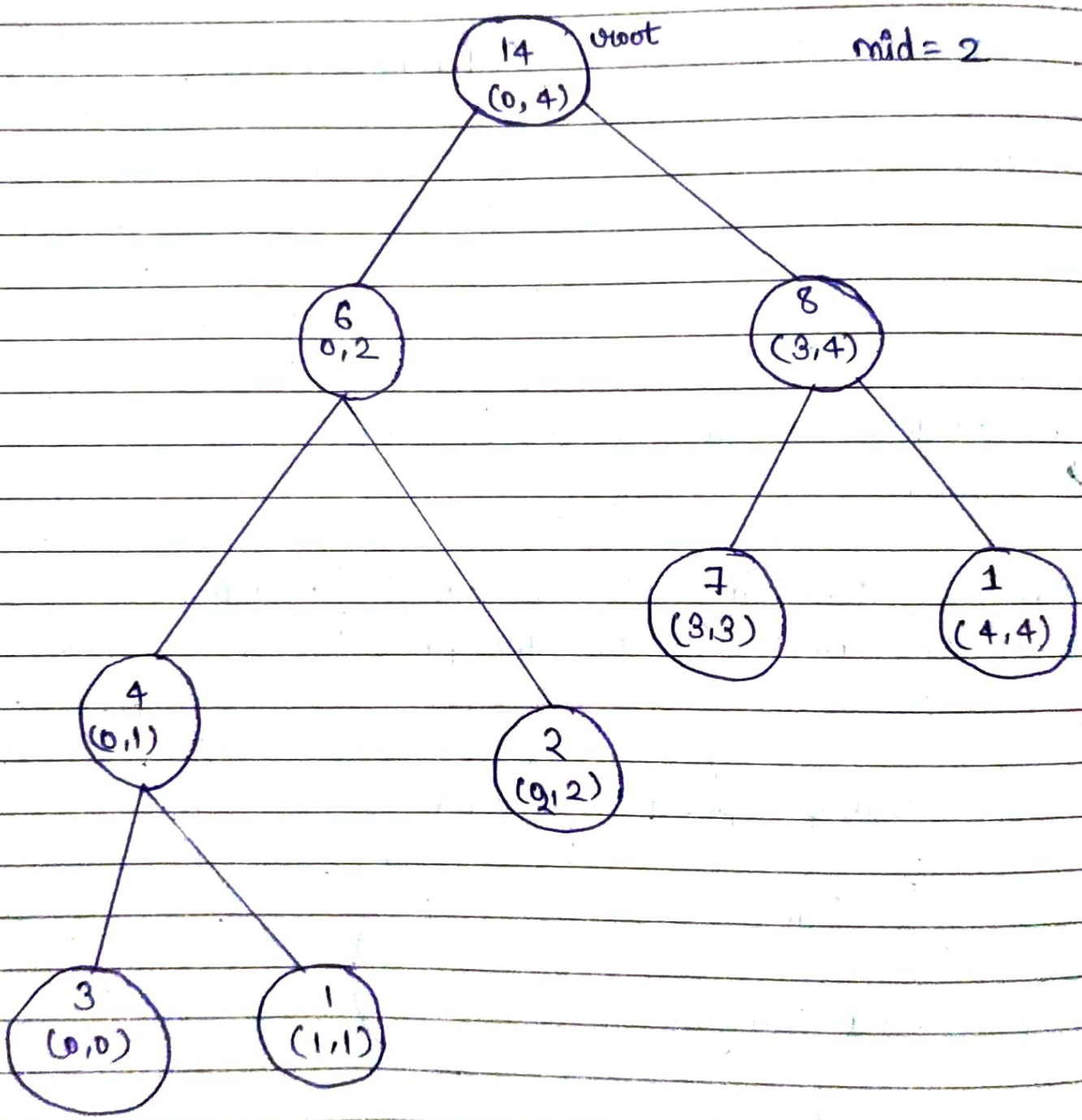
↳ maximum

↳ etc.

Date __/__/__
Intimate

⇒ How to Build Segment Tree ?

3	1	2	7	1
0	1	2	3	4



Date / / Intimate

01. Binary Tree
02. 2 children of all non-leaf nodes.
03. Leaf Nodes - Represents a single element in an array.
04. Root Node - Represents entire array
05. Other Nodes - Represents an interval or range of an array.

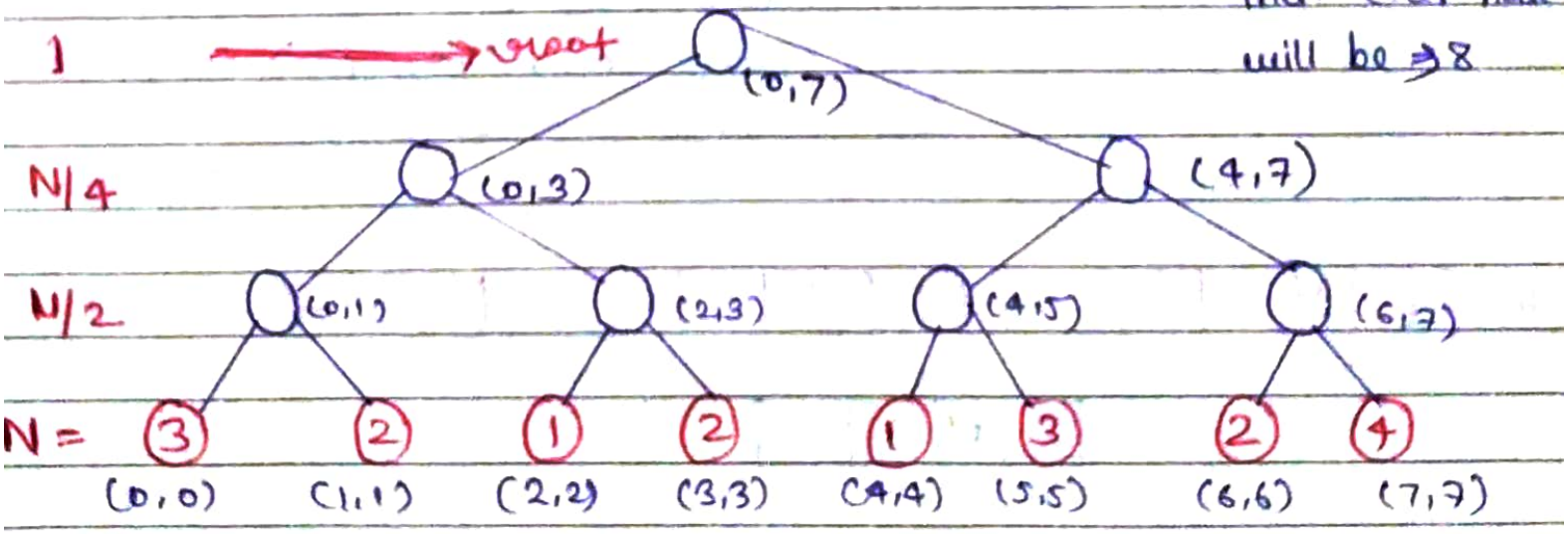
06. Height - $[\log_2(n)]$

07. Balanced Binary Tree.

⇒ How many Nodes -

0	1	2	3	4	5	6	7
3	2	1	2	1	3	2	4

$n = 8$
then leaf node will be ⇒ 8



nodes = $N + N/2 + N/4 + \cdots + 1$

↳ G.P

$$\text{Sum}_\infty = \frac{a}{1-r} = \frac{N}{1-\frac{1}{2}} = 2 * N$$

Date __/__/__
Intimate

Leaf node = N, Internal Node = (N-1)

$\approx$ N + N - 1 = (2N - 1) $\approx$ 2N

WHY AM I NOT USING PREFIX SUM ARRAY ?

	0	1	2	3	4
nums =	1	3	4	2	2

prefixSum =	1	4	8	10	12
	0	1	2	3	4

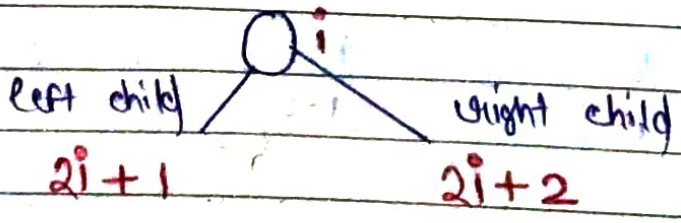
(1, 4)
⇒ 12 - 1
⇒ 11

① range sum ✓ possible
② range max-element ✗ not possible
③ range min-element ✗ not possible

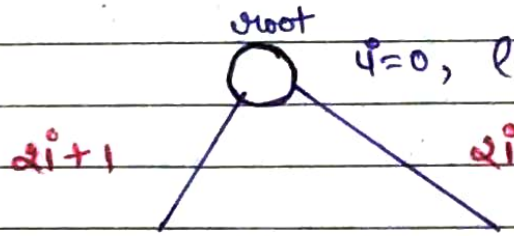
CODE FOR SEGMENT TREE -

3	1	2	7
0	1	2	3

Tree - Represent as an array



buildTree (0, l, n-1)



i=0, l=0, r=n-1=3, mid = $\frac{0+3}{2} \Rightarrow 1$

buildTree (2*i+1, l, mid)

buildTree (2*i+2, mid+1, r)

// Code:-

```
buildTree ( i, l, r ) {                                   or nums[l]
    // Base Case
    if ( l == r) { segTree[i] = nums[r]; return; }
    int mid = (l+r)/2;
    buildTree ( 2*i+1, l, mid );
    buildTree ( 2*i+2, mid+1, r);

    segTree[i] = segTree[2*i+1] + segTree[2*i+2];

    // size = 2*n
}
```

```
// Base Case
if (l == r) {                          or nums[l]
    segTree[i] = nums[r];
    return;
}
```

Date __/__/____ Intimate

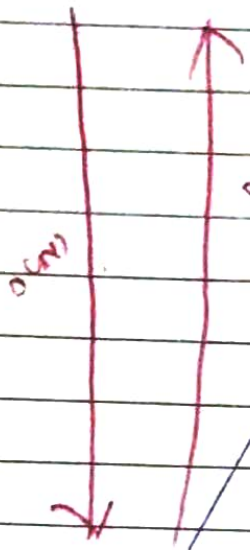
Recursion Dry Run:-

0	1	2	3
3	1	2	7

$n = 4$

i l r

bt (0, 0, 3) mid = (0+3)/2 ⇒ 1



bt (1, 0, 1) bt (2, 2, 3)

bt (3, 0, 0) bt (4, 1, 1) bt (5, 2, 2) bt (6, 3, 3)

l == r l == r l == r l == r

segTree =

0	1	2	3	4	5	6	
13	4	9	3	1	2	7	

...

Segment Tree array size = 2 * n

TC - O(2N)

SC - O(2*N) + O(log(N))

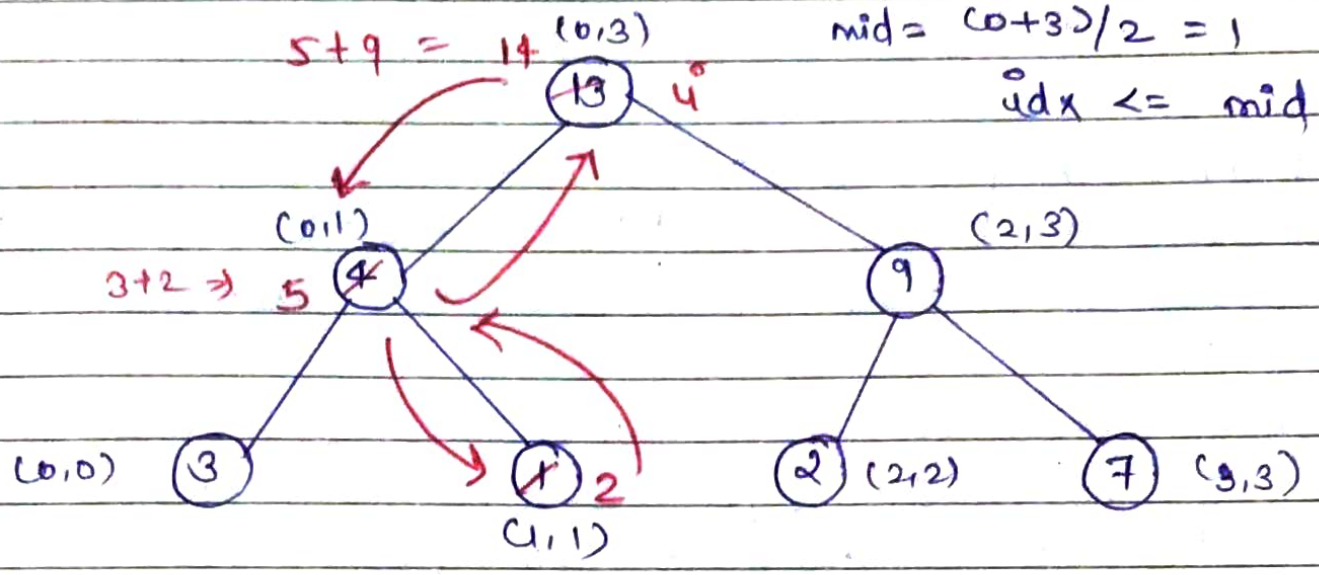
Prefix Sum technique से update query की time complexity - O(N)

Date __/__/__
Intimate

⇒ UPDATE QUERY -

3	1	2	7
0	1	2	3

update (idx = 1, val = 2)



Code :-

```
void updateSegTree (idx, val, i, l, r) {
    if (l == r) {          // Base Case
        segTree[i] = val;
        return;
    }
    mid = (l+r)/2;
    if( idx <= mid) { // left child
        updateSegTree (idx, val, 2i+1, l, mid);
    } else {          // right child
        updateSegTree (idx, val, 2i+2, mid+1, r);
    }

    segTree[i] = SegTree[2i+1] + SegTree[2i+2];

}
```

Total nodes update — log (N)

Date __/__/__
Intimate

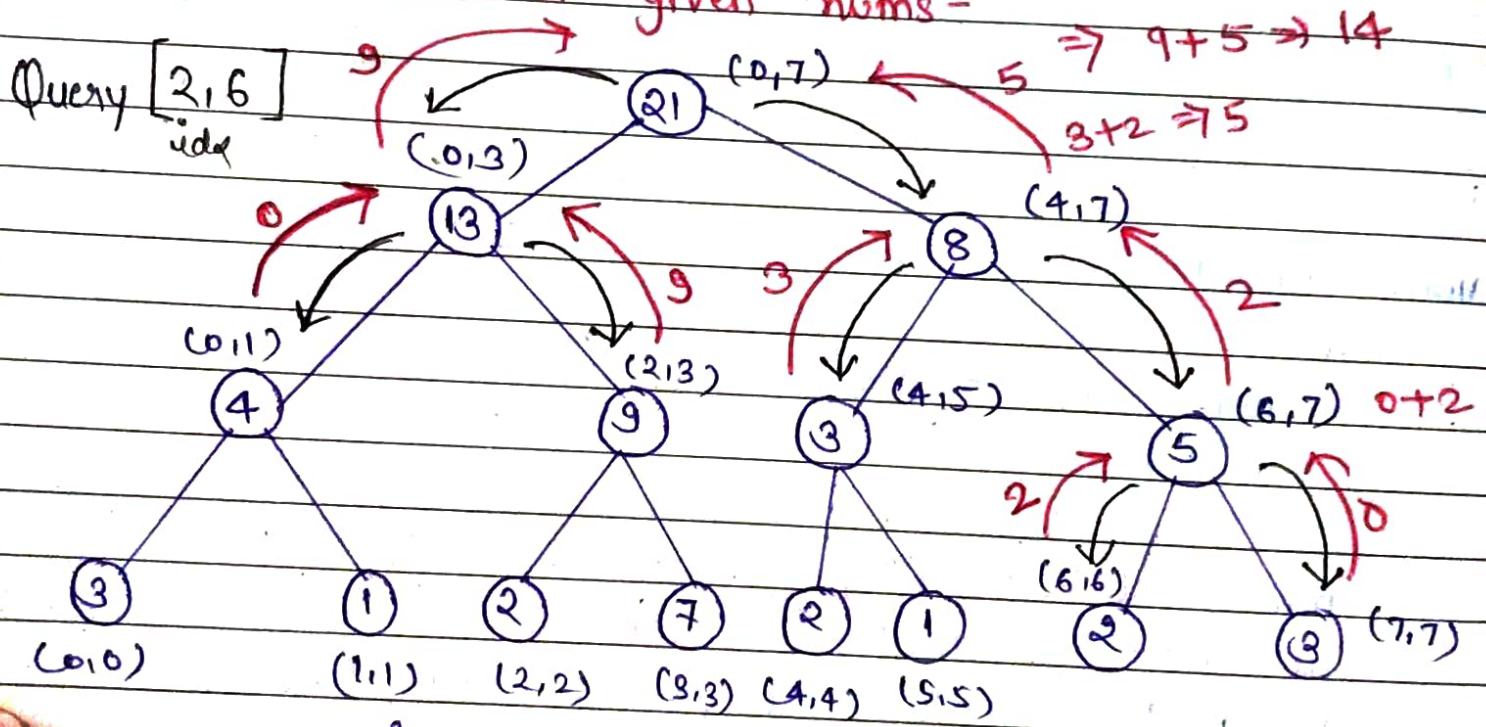
⇒ QUERY IN SEGMENT TREE -

Eample - Range Sum Query

nums = {3, 1, 2, 7, 2, 1, 2, 3}
0 1 2 3 4 5 6 7

Find Sum → [2,4] = 2+7+2
Find Sum → [1,5] = 1+2+7+2+1
Find Sum → [2,6] = ... etc.

Segment Tree of given nums -





① Out of bound → return 0

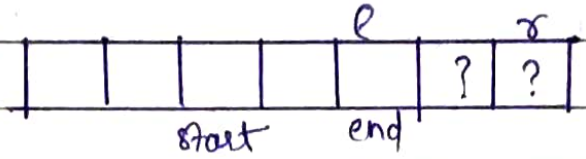
```
if (l > end || r < start){
    return 0;
}
```

Date __/__/__
Intimate

(2) [l, r] is entirely inside [start, end] →

```
if (l >= start && r <= end) {
    return segTree[i];
}
```

(3) else Overlapping –



return Query(leftValue) + Query(rightValue)

// Code: –

```
int Query(start, end, i, l, r) {
    if (l > end || r < start) { // out of bound
        return 0;
    }
    if (l >= start && r <= end) {
        return segTree[i];
    }

    return Query(start, end, 2i+1, l, mid) +
           Query(start, end, 2i+2, mid+1, r);
}
```

Time Complexity ≃ O((log(N)) * Q)

Space Complexity –

Date / / Intimate

⇒ WHY WE HAVE TO TAKE 4*N Size ARRAY FOR SEGMENT TREE -

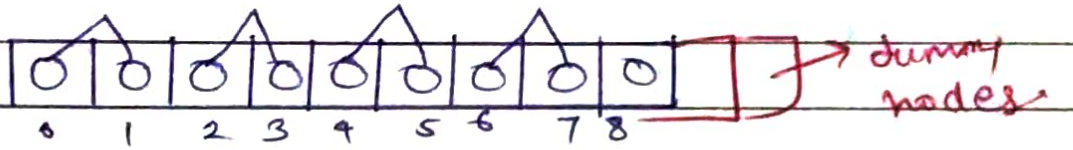
2^2 $n = 4 \rightarrow 7 \rightarrow (2^3 - 1)$

2^3 $n = 8 \rightarrow 15 \rightarrow (2^4 - 1)$

2^4 $n = 16 \rightarrow 31 \rightarrow (2^5 - 1)$

If n is not power of 2, we have to append some extra / dummy nodes.

Ex. $n = 9$



$(2 * N) + N + N/2 + N/4 + \ldots + 1$

$$\text{SumOfInf} = \frac{9}{1-r} = \frac{(2*N)}{1-\frac{1}{2}} = (4*N)$$

#

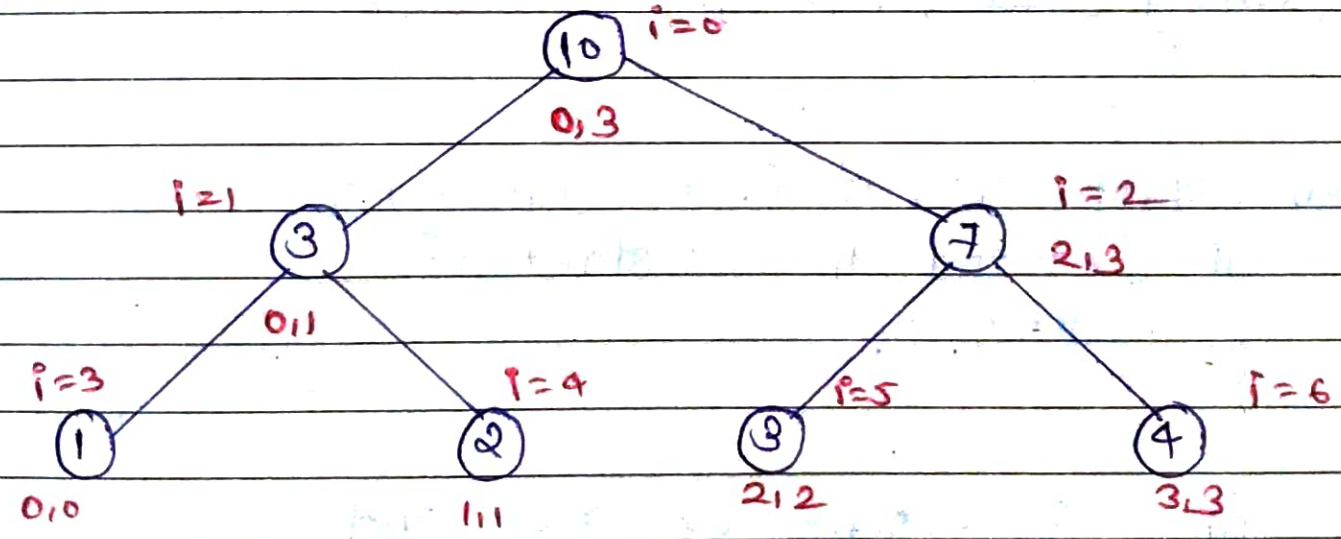
01. Query Sum II -

Example :- Input :- $n = 4$
arr = {1, 2, 3, 4}, $q = 2$
queries = {1, 4, 2, 3}

Output :- 10 5

Explaination - In the first ~~qur~~ query we need sum from 1 to 4 which is $1+2+3+4 = 10$.
In the second query we need sum from 2 to 3 which is $2+3 = 5$.

(1) BuildSegmentTree arr = {1, 2, 3, 4}



Code:-

```
void BuildT(int l, int r, int segTree, i){
    if (l == r){
        segTree[i] = arr[l];
        return;
    }

    int mid = l + (r-l)/2;

    BuildT(l, mid, segTree, 2*i+1);
    BuildT(mid+1, r, SegTree, 2*i+2);

    SegTree[i] = SegTree[2*i+1] + segTree[2*i+2];
}

{query(start, end, 0, 0, n-1, segTree);}

query(start, end, i, l, r, segTree){
    if (l > end || r < start){
        return 0;
    }

    if (l >= start && r <= end){
        return segTree[i];
    }
    mid = l + (r-l)/2;
    return query(start, end, 2i+1, l, mid, segTree)
        + query(start, end, 2i+2, mid+1, r, segTree);
}
```

Date __/__/__
Intimate

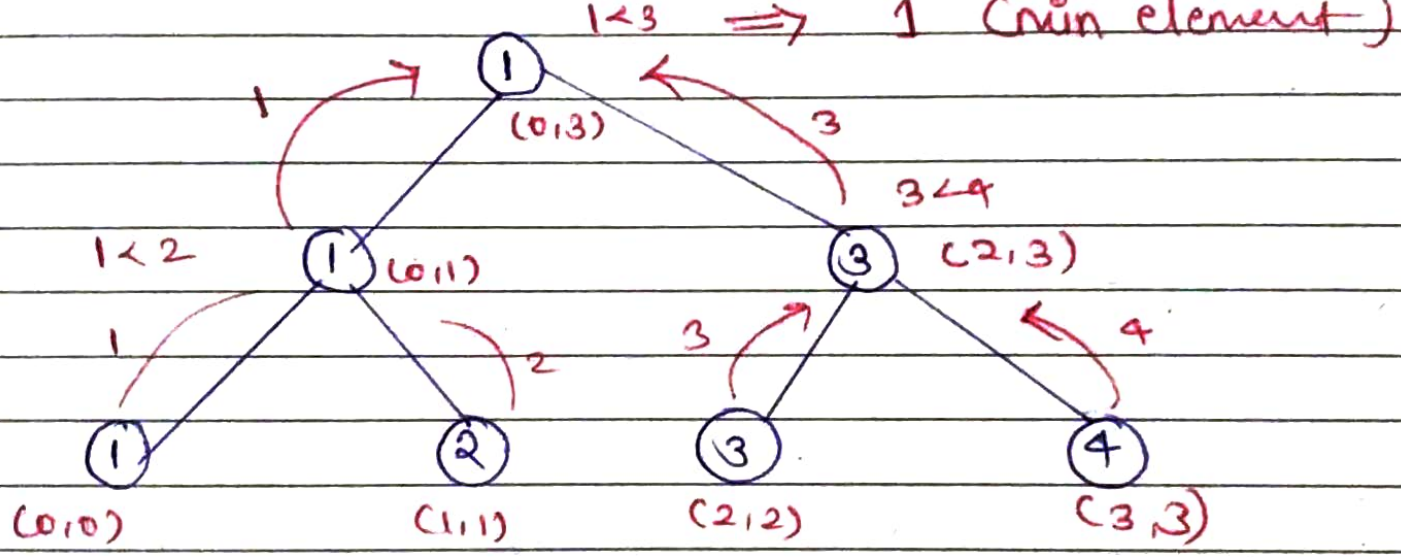
02. Range Minimum Query –

Input = arr = {1, 2, 3, 4}
0 1 2 3

RMQ = a = 0, b = 2

Output – 1

Segment Tree for given arr and problem –

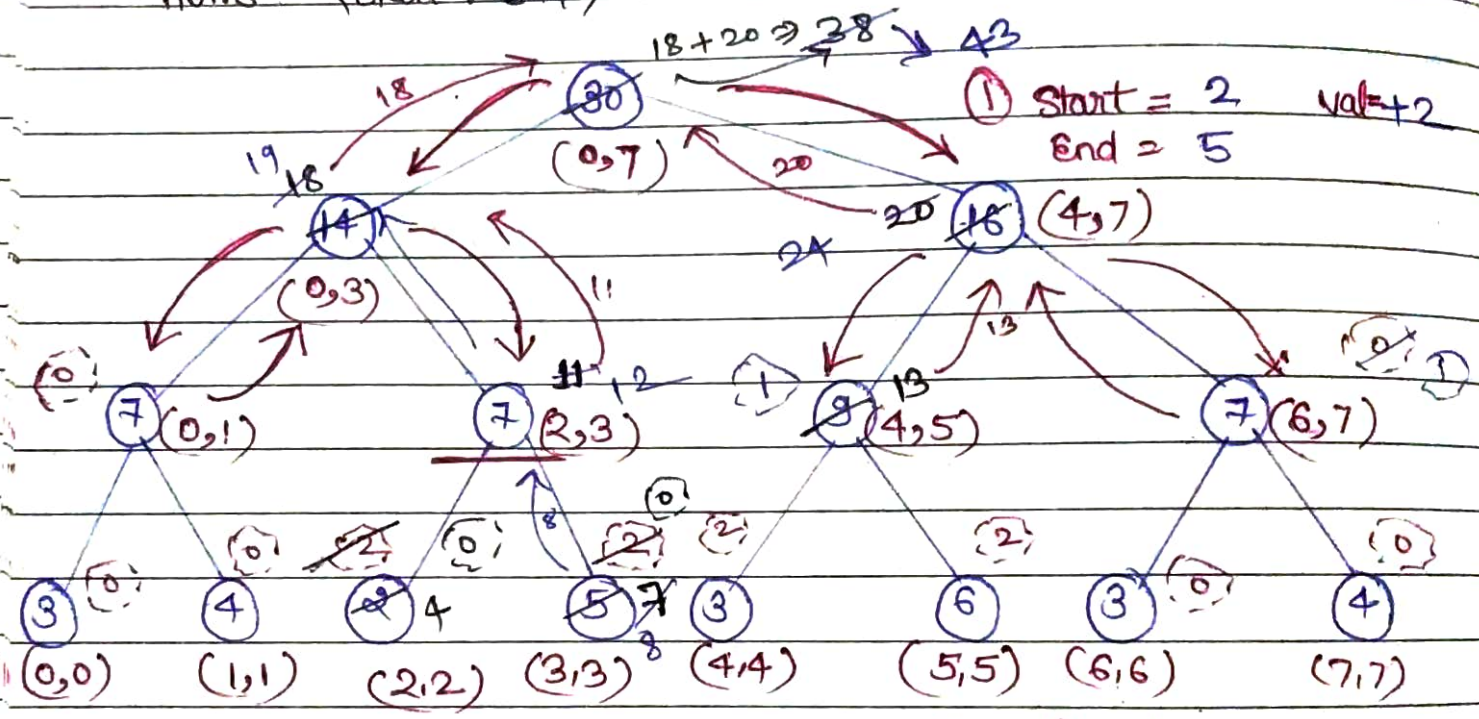


Date / /
Intimate

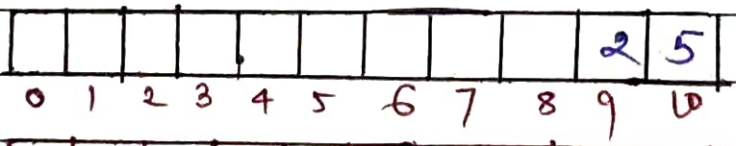
03. Range Update Query - (Lazy Propagation):-

nums = {3, 4, 2, 5, 3, 6, 3, 4}
0 1 2 3 4 5 6 7

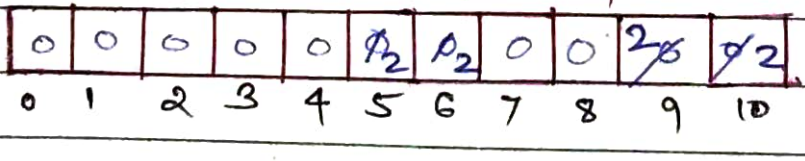
nums {start : end} ⟶ add +2



SegTree -



Lasy -



nums = {3, 4, 2, 5, 3, 6, 3, 4}
0 1 2 3 4 5 6 7

Code: -

```
void updateRange(int start, int end, int i, int l,
                 int r, int val, int segTree[],
                 int[] lazy) {
    // Apply any pending updates
    if (lazy[i] != 0) {
        segTree[i] += (r-l+1) * lazy[i];

        if (l != r) {
            lazy[2*i+1] += lazy[i];
            lazy[2*i+2] += lazy[i];
        }

        lazy[i] = 0;
    }
    // Out of range
    if (r < start || l > end || l > r) return;

    // Total Overlap
    if (start <= l && end >= r) {
        segTree[i] += (r-l+1) * val;

        if (l != r) {
            lazy[2*i+1] += val;
            lazy[2*i+2] += val;
        }

        return;
    }
```

Date __/__/__
Intimate

```
// partial overlap
int mid = (l+r)/2;
updateRange (start, end, 2*i+1, l, mid,
             val, segTree, lazy);

updateRange (start, end, 2*i+2, mid+1, r,
             val, segTree, lazy);

segTree[i] = segTree[2*i+1] + segTree[2*i+2];
}
```

TC - O(logN) → Height of the tree

SC - O(4*N) + O(4*N)

- O(4*N) → Segment Tree
- O(4*N) → Lazy Tree

Date / /
Intimate

04. Range Sum Query - Mutable

Example:- nums = {1, 3, 5}

Q_1 : sumRange (0, 2)

Q_2 : update (1, 2)

Q_3 : sumRange (0, 2)

Output:- Q_1 : 9

Q_3 : 8

Date __/__/__
Intimate

//Code:-

```
class NumArray {
    int n;
    int[] segmentTree;

    public NumArray(int[] nums) {
        n = nums.length;
        segmentTree = new int[4 * n];
        buildSegmentTree(0, 0, n-1, nums);
    }

    private void buildSegmentTree(int i, int l,
                int r, int nums[]) {
        if (l == r) {
            segmentTree[i] = nums[l];
            return;
        }

        int mid = l + (r-l)/2;
        buildSegmentTree(2*i+1, l, mid, nums);
        buildSegmentTree(2*i+2, mid+1, r, nums);

        segmentTree[i] = segmentTree[2*i+1] +
                         segmentTree[2*i+2];
    }

    public void update(int index, int val) {
        updateSegmentTree(0, 0, n-1, index, val);
    }
```

```
private void updateSegmentTree(int i, int l, int r,
                               int index, int val) {
    if (l == r) {
        segmentTree[i] = val;
        return;
    }

    int mid = l + (r-l)/2;

    if (index <= mid) {
        updateSegmentTree(2*i+1, l, mid, index,
                          val);
    }
    else {
        updateSegmentTree(2*i+2, mid+1, r,
                          index, val);
    }

    segmentTree[i] = segmentTree[2*i+1] +
                     segmentTree[2*i+2];
}

public int sumRange(int left, int right) {
    return query(0, 0, n-1, left, right);
}
```

```
private int query (int i, int l, int r,
                   int left, int right) {

    if ( right < l || left > r) {
        return 0;          // No overlap
    }

    if (left <= l && r <= right) {
        return segmentTree.ni); // Total overlap
    }

    int mid = l + (r-l)/2;

    return query(2*i+1, l, mid, left,
                 right) +

           query (2*i+2, mid+1, r,
                  left, right);

}

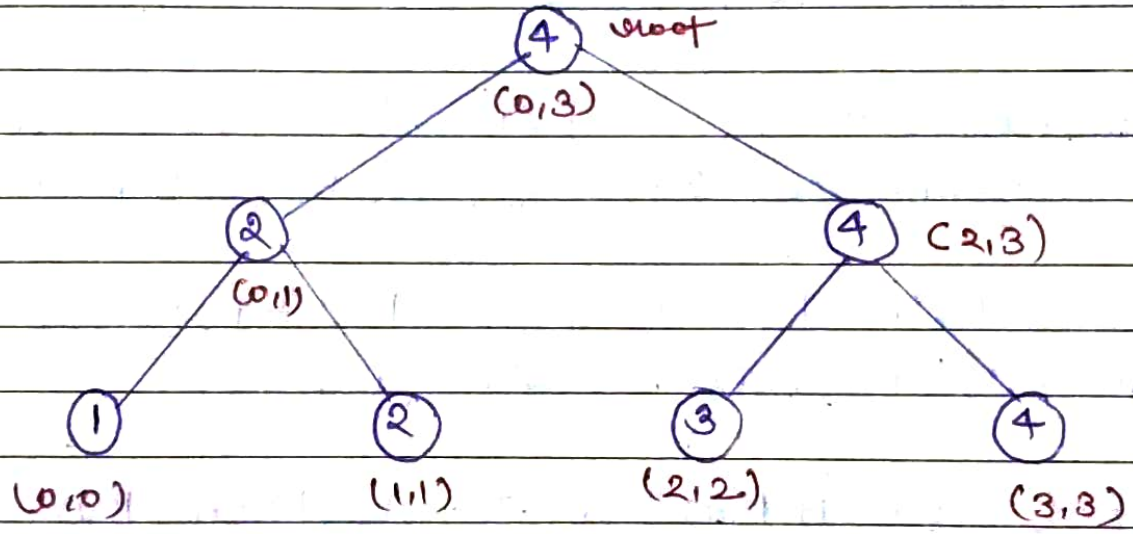
}
```

Date __/__/__
Intimate

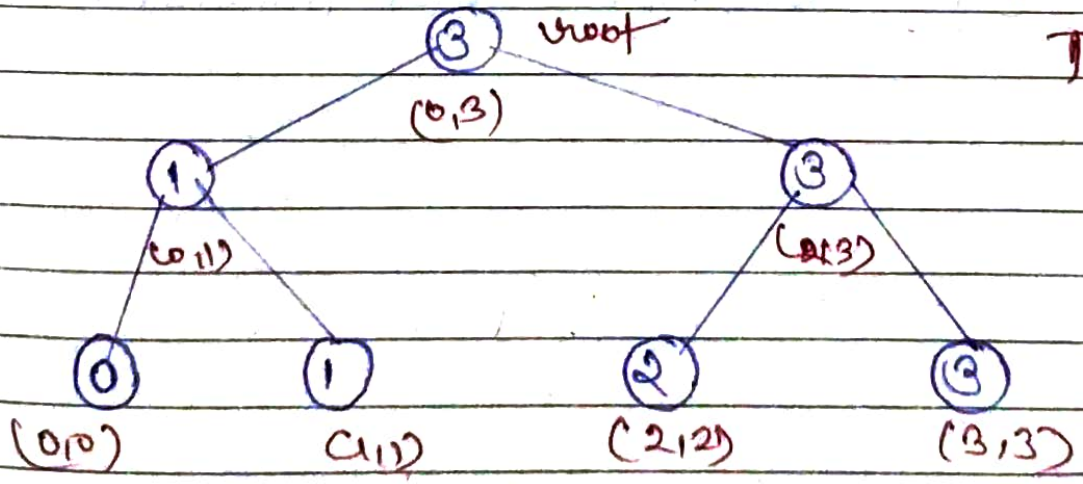
05. Range Maximum/ Index Query - Minimum

arr = {1, 2, 3, 4}
0 1 2 3

RMRQ = q = 0, b = 2 ; q = 2, b = 3 ; q = 1, b = 3



⇓ For Index



These nodes are index

Date __/__/__
Intimate

```
public void buildSegmentTree (int i, int l, int r,
                              segTree, arr) {

    if (l == r) {
        segTree[i] = l;
        return;
    }

    int mid = l + (r-l)/2;

    buildSegmentTree ( 2*i+1 , l, mid, segTree, arr);
    buildSegmentTree ( 2*i+2 , mid+1 , r, segTree, arr);

    int leftMaxIdx = segTree[2i+1];
    int rightMaxIdx = segTree[2i+2];

    if (arr[leftMaxIdx] >= arr[rightMaxIdx]){
        segmentTree[i] = leftMaxIdx;
    }
    else {
        segmentTree[i] = rightMaxIdx;
    }
}
```

```
int querySegTree(int start, int end, int i,
                 int l, int r, int[] segTree) {

    if (l > end || r < start) {   // invalid idx
        return -1;
    }

    if (l >= start && r <= end) {
        return segmentTree[i];
    }

    int mid = l + (r-l)/2;

    int leftMaxIdx = querySegTree(start, end,
                     2i+1, l, mid, segTree);

    int rightMaxIdx = querySegTree(start, end, 2i+2,
                      mid+1, r, segTree);
if(leftMaxIdx == -1) return rightmax.
if(rightmaxIdx == -1) return leftmaxIdx;
    if (arr[leftMaxIdx] >= arr[rightMaxIdx]) {
        return leftMaxIdx;
    }
    else {
        return rightMaxIdx;
    }
```

Date __/__/__
Intimate

☆ Binary Search In Range Queries

arr = { 5 , 3 , 7 , 4 , 1 , 6 }
0 1 2 3 4 5

Queries = { (0,1) , (0,2) , (1,3) , ... }

* Right Range में max element का index

{ 5 , 3 , 7 , 4 , 1 , 6 }
0 1 2 3 4 5

Query = (0,1)
⇓
RMIQ (x+1 , n-1)
(2 , 5)

* Right range में element index which is > arr[a] > arr[b]

0 1 2 3 4 5
{ 5, 3, 7, 4, 1, 6 }

a b
(0,1)

RMIQ → ~~2(idx)~~ → val =>

arr[idx] > arr[a] &&
arr[idx] > arr[b]

(01) RMIQ से greater ~~index~~ element's index निकालो

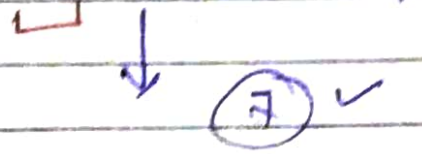
(2) Then Compare that greater element with Range index.

Date / /
Intimate

* Right range ke element index which is > arr[a] and > arr[b] and as left as possible

arr = { 0 5, 1 3, 2 7, 3 4, 4 1, 5 6 }

Query = (0, 1)



(7) ✓